Autonomous PID Parameter Tuning via Actor–Critic Reinforcement Learning

A Low-Cost Arduino-Based MIMO Temperature Control Testbed using TD3

Deepak Kumar Chauhan
Azaad Katiyar
Divy Kumar Pandey

1. Introduction and Motivation

We are proposing a project where we try to build a real, physical temperature control rig using Arduino and a heater, and then we will use a Reinforcement Learning agent to tune the PID gains of this system in real time. Normally in industry, PID tuning is done once using method like Ziegler–Nichols and then the gains is kept fixed for long time, but a real process may not be always staying same – it can drift because of environment change, sensor noise, or wear of component. We wants to check if a RL agent may adapt the gains on-the-fly, so that controller performance should stay good even when process characteristic is slowly changing. This project is combining both hardware (cheap and easily available parts) and software (a trained RL policy), and it may give a working demo of what otherwise is a purely simulation-based idea in most literature. Since we have not tried this on real rig before, all the results we mention in this proposal is only our expectation for now.

2. Objective

Our objective is to design and build a closed-loop temperature control system where:

1. An Arduino runs a fast, local PID loop which should keep the heater safe and stable at all time.
2. A Python program on laptop runs a trained TD3 (Twin Delayed DDPG) agent which observe the system state and decide new PID gains every few second.
3. The new gains is sent back to Arduino, which may update its own PID controller live, without stopping the control loop.

3. Data Flow – Step by Step (with analogy)

A easy way to think about our system: imagine Arduino as a *reflex nervous system* which is reacting fast and keeping the body (heater) safe every millisecond, while the RL agent on PC is like the *slow, thinking brain* which is watching the bigger picture and occasionally telling the reflex system to tune its values. Neither one is replacing the other, both are expected to work together.

Step 1 – Sensing: Every $\sim$200ms–1s, Arduino reads raw temperature (Process Value, PV) from the MAX6675 thermocouple amplifier.

Step 2 – Local error calculation: Arduino computes

$$\text{error} = \text{Setpoint} - \text{PV}$$

right there on the chip itself, so there should be almost no delay. This same error is used in two places. First, it goes in the temperature governing equation (the PID law), which have proportional, integral and differential term:

$$u(t) = K_p\, e(t) + K_i \int e(t)\, dt + K_d \frac{de(t)}{dt}$$

Second, it is also used by the RL agent for tuning the K_p, K_i, K_d values towards a optimal one. Because both the PID output and the gain tuning is depending on the same error, they may interact with each other and the system may diverge. To reduce this risk, we plan two thing: (i) we put limits (min and max) on the K_p, K_i, K_d updation, and (ii) the PID output and the gains is updated at different time interval (PID fast, RL slow). We think this should help, but we cannot say for sure before testing it on the real rig.

Step 3 – Local PID control (should never stop): Using whatever K_p, K_i, K_d is currently active, Arduino compute the PID output and drive the SSR (which switch the heater ON/OFF using PWM). This loop is designed to run continuously, even if RL agent is slow or laptop get disconnect – this is important for safety, heater should never just be uncontrolled.

Step 4 – Sending data to PC: Every cycle (roughly 1 sec), Arduino sends one line over Serial, something like:

```
T:78.4,SP:80.0,E:1.6,U:65
```

(Temperature, Setpoint, Error, current Output %).

Step 5 – Building the RL state (on PC): Python script reads this serial line and construct the state vector for the agent:

$$s_t = [\,\text{error},\ \text{error change},\ \text{past control output}\,]$$

Step 6 – Agent decides an action: The trained TD3 model should take s_t and output a continuous action:

$$a_t = [\Delta K_p,\ \Delta K_i,\ \Delta K_d]$$

which is added to current gains to get the new gains (within the safe limits).

Step 7 – Sending new gains back: PC sends the new gains back over Serial, like `K:2.5,0.8,0.1`.

Step 8 – Arduino updates PID: Arduino parse this string and call `myPID.SetTunings(Kp, Ki, Kd)` (from the standard `PID_v1` library) to live-update its control loop.

Step 9 – Repeat: This whole cycle (Step 1 to 8) keep repeating continuously, with the fast PID loop running independently at its own speed, and the slower RL update happening every 1–5 second. We expect this may be enough for meaningful adaptation without overloading the serial link, but the exact interval may need to be adjusted during testing.

4. Reinforcement Learning Algorithm: TD3

We are choosing **Twin Delayed Deep Deterministic Policy Gradient (TD3)** as our main RL algorithm for this project. TD3 is a actor-critic method, meaning it has two neural network: an Actor which maps the state directly to a continuous action (our $\Delta K_p, \Delta K_i, \Delta K_d$), and a Critic which evaluate how good that action really was.

TD3 is chosen instead of plain DDPG because DDPG is known to *overestimate* the Q-values, which can cause the Actor to learn towards actions which look good to the network but may not be actually good on the real system – quite risky for a heater which could overheat. TD3 try to fix this using three main idea:

- **Two critic networks** instead of one, and always taking the minimum of the two estimate, so the bias toward overestimation should get reduced.
- **Delayed actor update**, meaning critic network is updated more frequently than actor, so the critic may converge to more reliable value first.
- **Target policy smoothing**, where small noise is added to target action so the critic cannot easily exploit sharp, narrow peaks, this may keep the learned gain-adjustment smooth and not jerky.

Reward function we plan to use for training (and evaluation) is:

$$R_t = -|\text{error}| - \lambda(\Delta u_t)^2$$

where the second term penalize too much change in control output, so the agent should not learn to just slam the heater on/off violently to reduce error fast. The value of λ may need tuning.

5. Training of RL Model

We do not expect to need a huge dataset for training this agent. We plan to do 2–3 short step-tests on our actual Arduino rig (about 5 minutes each, holding heater at fixed % output and recording temperature response). This small amount of real data would be used only to fit three number (K, τ, θ – process gain, time constant, dead time) of a First Order Plus Dead Time model. After that, the TD3 agent may be trained inside a simulator built from this model, for around 300,000 timestep, which should cost almost nothing and put very little wear on the real hardware. Only after training is finish, the policy would be deployed back onto the real rig for testing. If the simple model is not matching the real rig well, the trained agent may not perform as expected and we may need to refit the model or retrain.

Hard safety bound is also planned on K_p, K_i, K_d (a min and max value each), and instead of simple clipping (which can cause the gradient to get stuck at boundary), we want to use the *inverting gradient* approach where gradient sign is flipped near the boundary, so the agent should learn to stay inside the safe zone. How well this works in practice, we will know only after experiment.

6. Expected Outcome

By end of this project we hope to have, but cannot guarantee:

- A working low-cost hardware rig demonstrating live PID gain adaptation.
- A trained TD3 agent (with DDPG as comparison baseline) that is expected to improve tracking performance under simulated drifting process condition.
- A short comparison of classical fixed-gain PID vs. our RL-adaptive PID on same rig, which may show improvement in settling time and overshoot, though the result can also come different from what we expect.